



Centre Européen de Formation

Assurez votre succès

PROJET DE FIN DE FORMATION

Application Web de Gestion de Tâches

Formation Développeur Web et Web Mobile (DWWM)

Présenté par : Céline Delecroix

Organisme de formation: Centre Européen de Formation

Session d'examen 2025 – 2026

Technologies utilisées

- HTML5
 - CSS3
 - JavaScript
 - Node.js
 - Express.js
 - MySQL
 - JWT
 - bcrypt
 - Font Awesome
-

**Projet réalisé dans le cadre de la validation du titre professionnel
Développeur Web et Web Mobile**

Remerciements

Je tiens à remercier le Centre Européen de Formation ainsi que l'ensemble des formateurs et intervenants qui m'ont accompagnée tout au long de ma formation Développeur Web et Web Mobile.

Je remercie également toutes les personnes qui m'ont apporté leur soutien et leurs conseils dans la réalisation de ce projet.

Ce projet m'a permis de mettre en pratique les connaissances acquises en développement Front-End et Back-End et de renforcer mes compétences dans la conception d'une application web complète.

Enfin, je remercie les membres du jury qui prendront le temps d'étudier ce dossier.

Sommaire

- Introduction
- Présentation du projet
- Technologies utilisées
 - Partie Front-End
 - Partie Back-End
 - Base de données
 - Architecture MVC
 - API REST
- Sécurité de l'application
 - Git et GitHub
- Tests de l'application
- Difficultés rencontrées
 - Conclusion

Introduction

Dans le cadre de ma formation Développeur Web et Web Mobile, j'ai réalisé une application web de gestion de tâches.

L'objectif de ce projet est de permettre à un utilisateur de créer un compte, se connecter de manière sécurisée et gérer ses tâches au quotidien grâce aux opérations CRUD (Create, Read, Update, Delete).

Ce projet m'a permis de mettre en pratique les compétences acquises durant ma formation, aussi bien en développement Front-End qu'en développement Back-End.

L'application a été développée à l'aide des technologies HTML, CSS, JavaScript, Node.js, Express.js et MySQL.

Une attention particulière a également été portée à la sécurité grâce à l'utilisation de JWT (JSON Web Token) et du hachage des mots de passe avec bcrypt.

Présentation du projet

L'application développée est un gestionnaire de tâches permettant aux utilisateurs de s'organiser et de suivre l'avancement de leurs activités.

Les principales fonctionnalités de l'application sont :

- Création d'un compte utilisateur ;
 - Connexion sécurisée ;
 - Ajout de nouvelles tâches ;
- Modification des tâches existantes ;
 - Suppression des tâches ;
 - Recherche de tâches ;
- Filtrage des tâches selon leur statut ;
 - Affichage des statistiques ;
- Barre de progression dynamique ;
 - Déconnexion sécurisée.

L'application repose sur une architecture MVC (Model View Controller) et communique avec une API REST développée avec Node.js et Express.js.

Les données sont stockées dans une base de données MySQL.

Technologies utilisées

Pour la réalisation de ce projet, plusieurs technologies ont été utilisées afin de développer aussi bien la partie Front-End que la partie Back-End de l'application.

Front-End

- HTML5 : structure des pages web ;
- CSS3 : mise en forme et design de l'application ;
- JavaScript : interactions et gestion dynamique des tâches ;
- Font Awesome : ajout d'icônes pour améliorer l'interface utilisateur.

Back-End

- Node.js : environnement d'exécution JavaScript ;
 - Express.js : création de l'API REST ;
 - MySQL : gestion et stockage des données ;
- JWT (JSON Web Token) : authentification sécurisée ;
 - bcrypt : chiffrement des mots de passe.

Outils utilisés

- Visual Studio Code ;
 - MySQL Workbench ;
 - Postman ;
 - Git ;
 - GitHub ;
 - Google Chrome.
-

Tableau récapitulatif des technologies

Tableau récapitulatif des technologies

Technologie	Utilisation
HTML5	Structure
CSS3	Style
JavaScript	Dynamisme
Font Awesome	Icônes
Node.js	Serveur
Express.js	API REST
MySQL	Base de données
JWT	Authentification
bcrypt	Sécurité
Git	Versions
GitHub	Sauvegarde
Postman	Tests
VS Code	Développement
Chrome	Vérification

Interface d'inscription

L'application propose une interface permettant à un nouvel utilisateur de créer un compte.

Le formulaire d'inscription demande :

- un nom d'utilisateur ;
- une adresse e-mail ;
- un mot de passe.

Ces informations sont ensuite envoyées au serveur Node.js afin d'être enregistrées dans la base de données MySQL.

Le mot de passe est sécurisé grâce à l'utilisation de bcrypt avant son enregistrement.

Capture d'écran de la page d'inscription



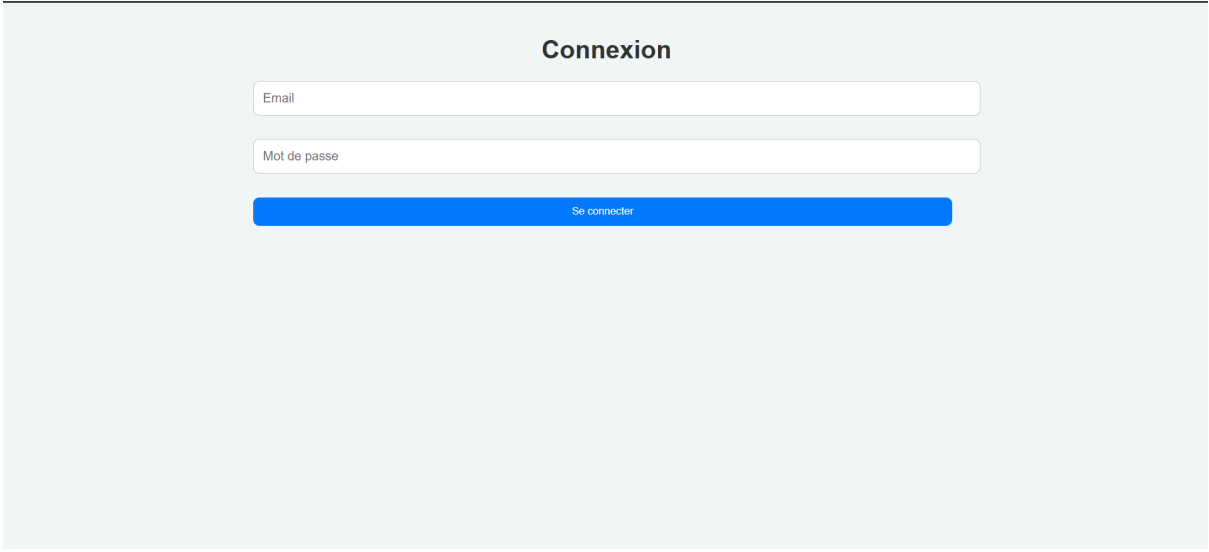
The screenshot displays a registration form titled "Créer un compte" centered on a light blue background. The form consists of three white input fields with rounded corners, each containing a placeholder label: "Nom", "Email", and "Mot de passe". Below these fields is a prominent blue button with the text "S'inscrire" in white. The entire form is centered horizontally and vertically within the light blue container.

Interface de connexion

L'utilisateur peut se connecter à son compte grâce à une interface sécurisée.

Après vérification des identifiants, un token JWT est généré et stocké dans le navigateur afin de permettre l'accès aux fonctionnalités de l'application.

Capture d'écran de la page de connexion

A screenshot of a web application's login page. The page has a light blue background. At the top center, the word "Connexion" is displayed in a bold, black font. Below this, there are two white input fields with thin gray borders. The first field is labeled "Email" in a small, gray font. The second field is labeled "Mot de passe" in a small, gray font. Below these fields is a solid blue button with the text "Se connecter" in white. The entire login form is centered on the page.

Dashboard principal

Une fois connecté, l'utilisateur accède au tableau de bord principal.

Cette interface centralise toutes les fonctionnalités de gestion des tâches.

Le dashboard permet :

- d'ajouter une nouvelle tâche ;
- de modifier une tâche existante ;
 - de supprimer une tâche ;
- de filtrer les tâches selon leur statut ;
 - de rechercher une tâche ;
- de consulter les statistiques de progression.

Capture d'écran du dashboard

The screenshot displays a web dashboard for task management. At the top, the title 'Mes tâches' is centered. Below it, a welcome message 'Bienvenue Céline' is followed by statistics: 'Nombre de tâches : 0', 'Tâches terminées : 0', and 'Progression : 0 %' with a corresponding progress bar. A red 'Déconnexion' button is present. The 'Ajouter une tâche' section includes input fields for 'Titre' and a description, and a blue '+ Ajouter' button. The 'Modifier une tâche' section has similar input fields and a blue 'Sauvegarder' button. Below these are filter buttons for 'Toutes', 'À faire', 'En cours', and 'Terminées', followed by a search bar labeled 'Rechercher une tâche...'. At the bottom, there is a link for 'Liste des tâches'.

Mes tâches

Bienvenue Céline

Nombre de tâches : 0
Tâches terminées : 0
Progression : 0 %

➔ Déconnexion

Ajouter une tâche

Titre

+ Ajouter

Modifier une tâche

Titre

Terminée

Sauvegarder

Filtrer les tâches

Toutes À faire En cours Terminées

Rechercher une tâche

Rechercher une tâche...

Liste des tâches

Gestion des tâches

L'application implémente les opérations CRUD :

- Create : création d'une tâche ;
- Read : affichage des tâches ;
- Update : modification d'une tâche ;
- Delete : suppression d'une tâche.

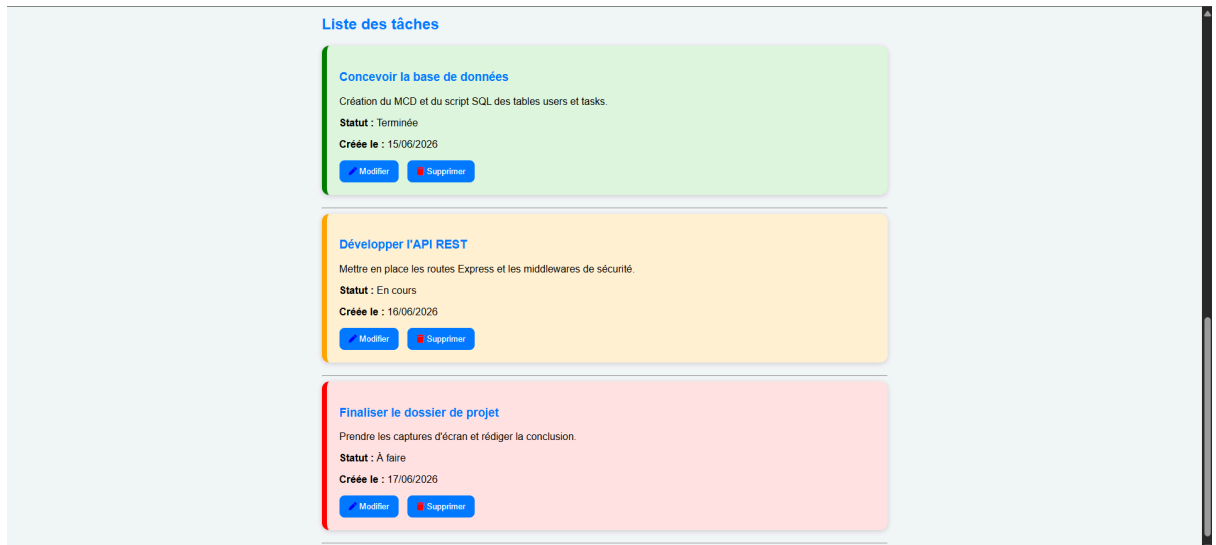
Chaque tâche possède :

- un titre ;
- une description ;
- un statut ;
- une date de création.

Les tâches sont affichées sous forme de cartes colorées afin d'améliorer la lisibilité.

Captures d'écran

- Ajout d'une tâche ;
- Modification d'une tâche ;
- Suppression d'une tâche.



Recherche, filtres et statistiques

Plusieurs fonctionnalités supplémentaires ont été mises en place afin d'améliorer l'expérience utilisateur.

L'utilisateur peut :

- rechercher une tâche grâce à un champ de recherche ;
- filtrer les tâches selon leur statut (« À faire », « En cours » ou « Terminée ») ;
 - visualiser le nombre total de tâches ;
 - consulter le nombre de tâches terminées ;
- suivre son avancement grâce à une barre de progression dynamique.

Captures d'écran

- Recherche d'une tâche ;
- Filtrage des tâches ;
 - Statistiques ;
- Barre de progression.

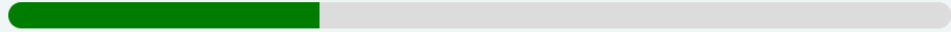
Mes tâches

Bienvenue Céline

Nombre de tâches : 3

Tâches terminées : 1

Progression : 33 %



🔒 Déconnexion

Rechercher une tâche

Liste des tâches

Développer l'API REST

Mettre en place les routes Express et les middlewares de sécurité.

Statut : En cours

Créée le : 16/06/2026

✎ Modifier

🗑 Supprimer

Filtrer les tâches

Toutes

À faire

En cours

Terminées

Rechercher une tâche

Liste des tâches

Concevoir la base de données

Création du MCD et du script SQL des tables users et tasks.

Statut : Terminée

Créée le : 15/06/2026

✎ Modifier

🗑 Supprimer

Présentation du Back-End (API REST)

Architecture globale du serveur

Le cœur logique de l'application repose sur un serveur **Back-End** développé avec l'environnement d'exécution **Node.js** et le framework **Express**. Ce choix technique garantit une architecture légère, hautement performante et asynchrone, idéale pour le traitement de requêtes HTTP rapides.

Le serveur fait office d'**API REST** (Representational State Transfer). Il reçoit les requêtes HTTP envoyées par le Front-End (navigateur), communique avec la base de données relationnelle pour récupérer ou modifier les informations, puis renvoie les réponses exclusivement au format **JSON**.

Structure du projet côté serveur

Pour assurer la maintenabilité et l'évolutivité du code, le projet Back-End suit une architecture modulaire stricte, inspirée du modèle **MVC (Modèle-Vue-Contrôleur)** :

- **server.js (ou app.js)** : Point d'entrée de l'application. Il initialise le serveur Express, configure les middlewares globaux (CORS, JSON parsing) et connecte la base de données.
- **Dossier /config** : Contient la configuration de la connexion à la base de données MySQL via le pilote (driver).
- **Dossier /routes** : Définit les points d'accès (endpoints) de l'API et associe chaque URL à une méthode de contrôleur spécifique.
- **Dossier /controllers** : Contient la logique métier (le traitement des données, les requêtes SQL, la validation).
- **Dossier /middlewares** : Regroupe les fonctions intermédiaires, notamment le contrôle de sécurité et la vérification des jetons d'accès.

Spécifications techniques des Routes et Sécurité

Tableau des points d'accès (Endpoints) de l'API

L'API REST expose plusieurs routes sécurisées permettant d'effectuer les opérations fondamentales du CRUD (Create, Read, Update, Delete) sur les tâches, ainsi que la gestion des utilisateurs.

Méthode HTTP	URL	Description	Authentification requise
POST	/api/auth/register	Inscription d'un nouvel utilisateur	Non
POST	/api/auth/login	Connexion de l'utilisateur (génère le Token)	Non
GET	/api/tasks	Récupération de toutes les tâches de l'utilisateur	Oui (JWT)
POST	/api/tasks	Création d'une nouvelle tâche	Oui (JWT)
PUT	/api/tasks/:id	Modification d'une tâche existante via son ID	Oui (JWT)
DELETE	/api/tasks/:id	Suppression d'une tâche via son ID	Oui (JWT)

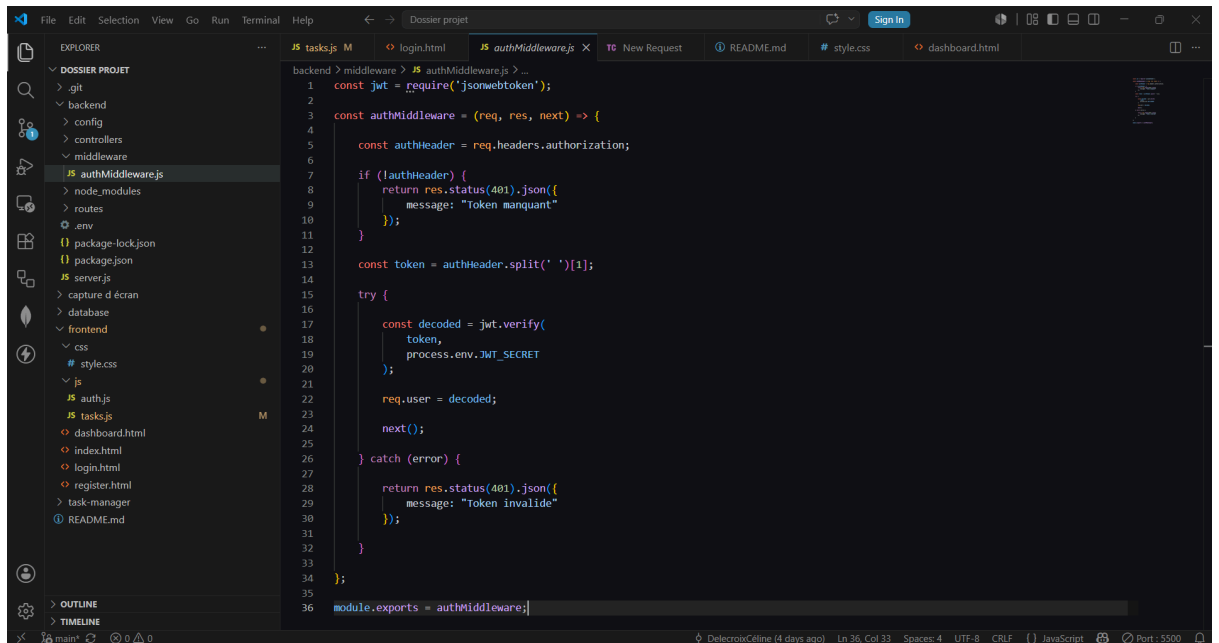
Sécurisation de l'API par Token (JWT)

La sécurité de l'application est un enjeu majeur. L'accès aux données des tâches est protégé par le protocole **JWT (JSON Web Token)**.

Lorsqu'un utilisateur s'authentifie avec succès sur la route [/api/auth/login](#), le serveur génère un jeton cryptographique signé à l'aide d'une clé secrète stockée dans les variables d'environnement (`process.env.JWT_SECRET`). Ce jeton contient l'identité chiffrée de l'utilisateur.

Pour chaque requête ultérieure vers les routes protégées (comme l'affichage ou l'ajout de tâches), le Front-End doit obligatoirement transmettre ce jeton dans l'en-tête de la requête HTTP sous la forme suivante : **Authorization: Bearer <TOKEN>**.

Un composant intermédiaire, l'**authMiddleware**, intercepte la requête côté serveur, extrait le jeton, vérifie sa validité et son expiration. Si le jeton est valide, la requête se poursuit ; s'il est invalide ou absent, le serveur bloque immédiatement l'accès et renvoie un code d'erreur **401 Unauthorized**.



```
1 const jwt = require('jsonwebtoken');
2
3 const authMiddleware = (req, res, next) => {
4
5   const authHeader = req.headers.authorization;
6
7   if (!authHeader) {
8     return res.status(401).json({
9       message: "Token manquant"
10    });
11  }
12
13  const token = authHeader.split(' ')[1];
14
15  try {
16
17    const decoded = jwt.verify(
18      token,
19      process.env.JWT_SECRET
20    );
21
22    req.user = decoded;
23
24    next();
25  } catch (error) {
26
27    return res.status(401).json({
28      message: "Token invalide"
29    });
30  }
31
32 }
33
34 module.exports = authMiddleware;
```

Structure de la Base de Données

Présentation de la base de données

Le stockage des informations est confié à **MySQL**, un système de gestion de base de données relationnelle (SGBDR) robuste, performant et largement utilisé dans l'industrie. La persistance des données garantit que les utilisateurs retrouvent leurs tâches intactes à chaque nouvelle connexion.

Schéma des tables (Modèle Relationnel)

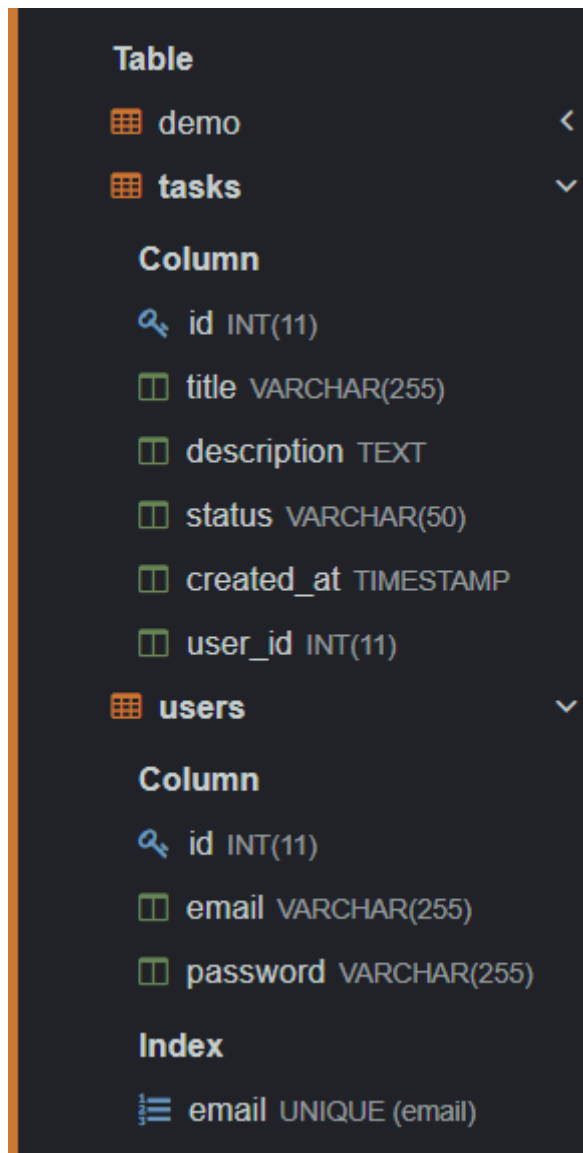
La base de données est structurée autour de deux tables principales liées par une relation de clé étrangère (un utilisateur possède plusieurs tâches) :

Table `users` (Gestion des comptes utilisateurs)

- `id` (INT, Clé Primaire, Auto-incrémenté) : Identifiant unique de l'utilisateur.
- `email` (VARCHAR, Unique) : Adresse e-mail servant d'identifiant de connexion.
- `password` (VARCHAR) : Mot de passe de l'utilisateur (haché pour des raisons de sécurité).

Table `tasks` (Gestion des tâches)

- `id` (INT, Clé Primaire, Auto-incrémenté) : Identifiant unique de la tâche.
 - `title` (VARCHAR) : Titre ou nom de la tâche.
 - `description` (TEXT) : Détails optionnels de la tâche.
- `status` (VARCHAR) : État actuel de la tâche (« À faire », « En cours », « Terminée »).
- `created_at` (TIMESTAMP) : Date et heure automatiques de création de la tâche.
- `user_id` (INT, Clé Étrangère pointant vers `users.id`) : Identifie l'utilisateur propriétaire de la tâche.



The screenshot displays a database interface with a dark theme. It shows two tables: 'demo' and 'tasks'. The 'tasks' table is selected, and its columns are listed: 'id' (INT(11)), 'title' (VARCHAR(255)), 'description' (TEXT), 'status' (VARCHAR(50)), 'created_at' (TIMESTAMP), and 'user_id' (INT(11)). Below the 'tasks' table, the 'users' table is also visible, with columns 'id' (INT(11)), 'email' (VARCHAR(255)), and 'password' (VARCHAR(255)). An index is shown for the 'users' table on the 'email' column, labeled 'email UNIQUE (email)'.

Table	
demo	<
tasks	▼
Column	
id INT(11)	
title VARCHAR(255)	
description TEXT	
status VARCHAR(50)	
created_at TIMESTAMP	
user_id INT(11)	
users	▼
Column	
id INT(11)	
email VARCHAR(255)	
password VARCHAR(255)	
Index	
email UNIQUE (email)	

Conclusion et Perspectives

Bilan du projet

Le développement de cette application de gestion de tâches (Task Manager) a permis de concevoir et de déployer une application web complète de bout en bout (Full-Stack).

L'objectif principal a été atteint : proposer une interface moderne, fluide et réactive (Front-End) tout en garantissant la sécurité, l'étanchéité des données par utilisateur et la robustesse des traitements côté serveur (Back-End). Ce projet a permis de consolider des

compétences clés en développement JavaScript moderne (ES6+), manipulation asynchrone des API ([fetch](#), [async/await](#)), architecture MVC, gestion de bases de données relationnelles SQL et mise en œuvre de protocoles de sécurité standardisés (JWT).

Difficultés rencontrées et solutions

Au cours du développement, plusieurs défis techniques ont dû être surmontés :

- **La gestion des requêtes asynchrones** : S'assurer que l'interface graphique ne se mette à jour qu'une fois que la base de données a bien validé l'action. L'utilisation rigoureuse des blocs [try/catch](#) et de [async/await](#) a permis de fluidifier ces processus.
- **Le maintien de la session utilisateur** : La perte des jetons d'authentification lors de l'actualisation des pages a été corrigée en exploitant judicieusement le mécanisme de stockage local ([localStorage](#)) du navigateur.
- **Les conflits d'infrastructures locales** : Les caprices de démarrage de services locaux (conflits de ports ou altération des fichiers temporaires MySQL lors des interruptions du système) ont mis en évidence l'importance d'intercepter correctement les codes d'erreurs HTTP (comme les statuts 401 et 500) pour préserver l'expérience utilisateur, même en cas de défaillance du serveur.

Perspectives d'évolution

Afin d'aller plus loin, plusieurs fonctionnalités de second plan pourraient être intégrées dans une future version :

- **Ajout de fonctionnalités collaboratives** : Permettre le partage d'une liste de tâches entre plusieurs utilisateurs avec un système de commentaires.
 - **Gestion des dates d'échéance** : Intégrer un calendrier et un système de notifications par e-mail à l'approche de la date limite d'une tâche.
 - **Mode Sombre (Dark Mode)** : Améliorer le confort visuel des utilisateurs via une option d'accessibilité dans l'interface CSS.
- **Déploiement en ligne** : Héberger l'API sur une plateforme Cloud (comme Render ou Heroku) et la base de données sur un service managé pour rendre l'application accessible partout dans le monde.